Assumptions & Decisions Log

Inside Airbnb — New York City | Supplementary Deliverable (Assignment Section 11.1)

Candidate Project	Inside Airbnb Market Intelligence - New York City
Scope Strategy	Single city, depth-focused (vs. multi-city breadth)
Companion Documents	Professional PDF Report · Summary: Completed Work · Summary: Incomplete Work · AI Usage Disclosure
Source Notebooks	01_dataset_familiarization.ipynb · 03_Data_Engineering_Challenges.ipynb · 04_Exploratory_Data_Analysis_.ipynb

This document consolidates every assumption made about ambiguous or incomplete data, and every significant engineering decision taken during the assignment, into a single reference. It is extracted from Sections 2.2–2.3, 3.6, 5.7 and 15.2 of the main report so that reviewers can audit the candidate's reasoning without reading the full 25-page document. Each entry follows the format the assignment brief requests: **what was considered**, **what was chosen**, and **what trade-off was accepted**.

A. Strategic & Scope Decisions

Two decisions were made before any code was written, because they determined the shape of every decision that followed.

A.1 City Selection

Element	Detail
Options considered	Single city (depth-first) vs. 2–6 cities (breadth, comparative analytics)
Chosen approach	New York City only — single-city, depth-focused strategy
Why	NYC is the largest, most mature market in the Inside Airbnb catalogue, with the widest variety of property types, pricing dynamics and neighbourhood-level heterogeneity -the richest single environment in which to demonstrate analytical depth.
Trade-off accepted	No cross-city comparative insight or harmonisation experience is demonstrated directly (see Summary: Incomplete Work, Section 12). The assignment's own guidance - “one city analyzed with exceptional depth beats five cities skimmed superficially” - was taken at face value.

A.2 Section Prioritisation Ordering

Given the assignment is deliberately over-scoped, the following priority ordering was applied before work began:

Priority	Section	Rationale
1 - Mandatory	02 Dataset Familiarization	Required foundation; every downstream decision depends on understanding the data first.
2 - Core Value	03 Data Engineering Challenges	Highest single weight in the evaluation rubric (25%); demonstrates engineering rigor directly.
3 - Recommended	04 Exploratory Data Analysis	Second-highest combined rubric weight; converts engineering work into business value.

Priority	Section	Rationale
4 - Deferred	Sections 05–08	Statistical, ML, AI/LLM and open-innovation sections; time-constrained and optional/recommended rather than mandatory; covered with structural commentary only (see Summary: Incomplete Work).

B. Dataset Assumptions

The Inside Airbnb data dictionary leaves several fields open to interpretation. The following assumptions were made explicit before any cleaning or analysis proceeded, per Section 3.6 of the report.

- **Calendar availability convention.** *available* = *'f'* in calendar.csv is treated as *booked* (not host-blocked) for occupancy estimation. This follows the Inside Airbnb convention, although blocked and booked nights are not distinguishable in the source data - so occupancy estimates carry a known directional bias (documented in Limitations & Caveats).
- **Price snapshot, not average.** The *price* field in listings.csv is treated as the most recent night's price at scrape time, not a time-averaged price. Calendar.csv in this scrape does not carry a per-date price, so no seasonal price series can be reconstructed from it.
- **Zero-review listings are not excluded.** Listings with 0 reviews may be legitimately new or simply inactive. Rather than drop them, they are retained and flagged with *has_reviews* = *False* so downstream analyses can choose to include or exclude them explicitly.
- **Out-of-bounds coordinates retained, not dropped.** 27 listings fall slightly outside the NYC bounding box. These are kept with a flag rather than deleted, on the assumption that they may legitimately serve the NYC market from adjacent municipalities.
- **Missing structural fields use group-median imputation.** Bedrooms, bathrooms and beds carry 31–36% null rates. Three imputation strategies were evaluated; group-median imputation by *room_type* × *accommodates* (“Strategy B”) was adopted as the production approach, paired with a boolean *was_missing* flag on each affected column so the imputation remains visible and reversible rather than silently baked into the data.

C. Engineering Decision Log

For every significant tool-selection or architectural choice, the options considered, the approach chosen, and the trade-off knowingly accepted are recorded below (Section 5.7 of the report).

Decision	Options Considered	Chosen Approach	Trade-off Accepted
Database engine	PostgreSQL, SQLite, DuckDB	DuckDB for the analytical star schema; SQLite for pipeline metadata	DuckDB does not enforce foreign-key constraints on existing tables; mitigated through documented join logic instead of database-enforced referential integrity.
Missing value strategy	Drop rows, impute median, sentinel value	Group-median imputation with was_missing flags	Introduces imputed values that differ from the (unknown) true values; the was_missing flag preserves full transparency about which cells were estimated.
Outlier handling	Hard drop, soft flag, leave in	Flag and retain; hard-drop only for clear domain violations	Outliers remain in the dataset by default; analysts must filter deliberately rather than relying on pre-removal, which keeps legitimate luxury/edge-case listings visible.
Pipeline configuration	YAML file, environment variables, inline Python dict	Inline CITY_CONFIG dictionary	Less flexible than YAML for true multi-environment deployment, but clearer and faster to read inside a single notebook-based assignment context.

Supplementary tool-selection rationale (Section 4.2):

Tool	Why Selected	Alternative Considered
Python + pandas	Most widely used data analysis stack; team-transferable; rich ecosystem	R -equally capable but narrower adoption in engineering roles
DuckDB	Zero-server analytical SQL engine; runs in-process; ideal for single-node work	PostgreSQL (requires server setup) · SQLite (limited analytical SQL)
SQLite (metadata)	Lightweight, zero-dependency, file-based; ideal for pipeline metadata tracking	JSON file (not queryable) · in-memory dict (not durable)
matplotlib + seaborn	Fine-grained control over publication-quality static figures	Plotly - interactive, but harder to embed in a static PDF report
MD5 checksums	Idempotent incremental processing - reprocess only when content changes	File timestamps - unreliable across operating systems and version control

D. Strategic Trade-offs Accepted

Beyond individual tool choices, four higher-level trade-offs shaped the whole submission (Section 15.2, Reflection).

Trade-off	Decision	Rationale
Depth vs. breadth	1 city, analyzed deeply - over 5+ cities, analyzed shallowly	The assignment brief is explicit that depth outperforms breadth. NYC alone offers enough structural richness (boroughs, regulation, market segments) for a thorough demonstration.
DuckDB vs. PostgreSQL	DuckDB used for the assignment; PostgreSQL noted as the production recommendation	DuckDB removes server-setup friction for a one-week, single-analyst project. The star-schema design itself is fully portable to PostgreSQL if the project moved to production.
Custom quality scoring vs. Great Expectations	Built a bespoke quality-scoring framework	Great Expectations is the production-correct choice but adds non-trivial setup overhead. A custom framework demonstrates equivalent reasoning (completeness / validity / uniqueness / consistency) with far less tooling dependency.
Occupancy-rate bias	Accepted the known bias and documented it clearly rather than attempting a silent correction	The bias (booked vs. host-blocked nights being indistinguishable) is structural to the data source, not a pipeline defect. Transparent disclosure is the correct response; inventing a correction without ground truth would be methodologically worse.